ACPC Notes — Contents

1 Contest Playbook **1**

1.1 ACPC, Kattis, and Team Strategy 1

1.2 Template, Debugging, Stress Testing 2

1.3 When You're Stuck 3

1.4 ACPC 2016–2024: Every Problem → Technique 6

1. Contest Playbook

1.1 ACPC, Kattis, and Team Strategy

How to use these notes. Stuck on a problem → go to *When You're Stuck* (§1.3): constraint → complexity table, then keyword → technique table, which points to the section with code. Every C++ block compiles as-is after the template (§1.2) with `#include <bits/stdc++.h>`, `using namespace std;`, `using ll = long long;`. Blocks marked **[stress-tested]** were checked against brute force; the rest compile and were hand-verified, so still test on samples.

1.1.1 What ACPC looks like (from all sets 2016–2024 on Kattis)

- 8–15 problems per set. Kattis difficulty (1–10 scale) spread is very consistent:
- **~35% easy (1.2–2.9):** implementation, careful reading, simulation. Solve these *fast and without WA* — penalty time decides ranks among teams with equal solves.
- **~35% medium (3.0–5.0):** one standard idea: binary search on answer + greedy, monotonic stack, Dijkstra/BFS with a twist, tree DP, counting mod p, interval DP, grid flood fill, offline DSU.
- **~30% hard (5.1–8.3):** probability/expected-value DP, flows with modelling (min flow with lower bounds, assignment), FFT, number theory (factoring, gcd tricks), interactive problems with tiny query budgets, geometry + search, string canonical forms, nasty interval DP.
- Expert/CM level: easy+medium ≈ CF Div2 A–C; hard ≈ Div2 D–F / Div1 C–D. Hard ACPC problems are usually *one known technique plus modelling* rather than research-level tricks — recognizing the model is the whole game. See §1.4 for every past problem mapped to its technique.
- Themes recur: Kattis "Medium/Hard" problems from ACPC very often involve **probability**, **flows/matching**, **Dijkstra on augmented state**, **binary search on the answer**, and **number theory**.

1.1.2 Kattis specifics

- Language: C++ compiled with GCC, `-O2`, gnu++17 or newer, 64-bit. `__int128`, `__builtin_*`, `<bits/stdc++.h>`, `__gnu_pbds` all work.
- Time limit is **per test file**; there may be dozens of files. Memory is usually 1024 MB (check the problem page).
- Verdicts stop at the first failing test file — you don't learn which case failed. A WA on test 1 means the sample itself fails (or format is wrong).
- Output: trailing whitespace/newline differences are ignored. Floating answers are accepted within the stated absolute/relative error; print with `fixed << setprecision(10)`.
- **Interactive problems** (ACPC 2024 had two): flush after every query (`cout << ... << endl` or `cout.flush()`), never read past what the judge sends, terminate immediately after the final answer. See §10.1.
- Multiple test cases inside one file (`T` given): **reset all global state**.
- Python: big integers are trivial in Python; consider it for one-off big-number problems (e.g. ACPC 2017 *Divide by 100* has numbers with 10^6 digits — just string manipulation though).
- Recursion: Kattis has a generous stack, but DFS depth 10^6 in C++ is still risky; prefer iterative for path graphs with n ≥ 10^6.

1.1.3 Team strategy (2–3 people, one machine)

1. **First 15 min:** split the problem set; everyone reads, labels each problem `easy / medium / hard / unknown` and writes one-line summary + constraints on paper. Read *all* problems — ACPC difficulty is not ordered by letter.
2. **Keyboard discipline:** only one person types; others think on paper. Never debug at the keyboard while someone with a ready solution waits — print/write the code and debug on paper.
3. Watch the scoreboard: problems solved by many teams are easier than they look. If 10 teams solved X and you think it's hard, you're missing something simple.
4. **Before submitting:** run all samples, a tiny edge case (n = 1, all equal, max values), and check output format. One WA costs 20 minutes of penalty.
5. **After WA:** reread the statement *first* (most WAs are misreads), then check §1.3 bug list, then write a brute force and compare on random small inputs.
6. **Last hour:** stop starting new hard problems unless the idea is complete; finish/debug what's closest. Freeze-time submissions still count.
7. Two people can pair on the hardest problem: one explains the problem from scratch to the other — this exposes wrong assumptions.

1.1.4 Precision and number sizes (memorize)

type	max	notes
`int`	2.1·10^9	`a*b` overflows when both ~ 5·10^4
`long long`	9.2·10^18	product of two 10^9 values fits; two 10^10 doesn't
`__int128`	1.7·10^38	multiply two `ll` safely; no I/O, write a printer
`double`	15–16 digits	integers exact up to 2^53 ≈ 9·10^15
`long double`	18–19 digits	80-bit on x86 GCC; slower but helps precision

- Operations/second: ~ 3·10^8 simple ops in 1 s with `-O2`; `set/map` ~ 10^7 ops/s; `unordered_map` ~ 3·10^7 with reserve; `cin` with sync off ~ 10^7 ints/s.
- Memory: `int` array of 10^8 = 400 MB. `vector<vector<int>>` with 10^6 small vectors has big overhead — use CSR / flat arrays.

1.2 Template, Debugging, Stress Testing

1.2.1 Main template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
#ifdef LOCAL
#define dbg(...) cerr << "[" << #__VA_ARGS__ << "] = ", _dbg(__VA_ARGS__)
template <class T> void _dbg(const T& x) { cerr << x << endl; }
template <class T, class... R> void _dbg(const T& x, const R&... r) {
    cerr << x << ", "; _dbg(r...);
}
#else
#define dbg(...)
#endif
const ll INF = 4e18;  // careful: INF + INF overflows. Use 1e18 if you add two.
const int MOD = 1e9 + 7;

void solve() {
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    int T = 1;
    // cin >> T;
    while (T--) solve();
}
```

Compile locally: `g++ -std=gnu++17 -O2 -Wall -Wextra -DLOCAL -fsanitize=address,undefined -g a.cpp`. Sanitizers catch out-of-bounds and signed overflow immediately — use them whenever you get RE/WA.

1.2.2 Useful snippets

```cpp
// print __int128
void print128(__int128 x) {
    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) print128(x / 10);
    putchar('0' + (int)(x % 10));
}
// floor/ceil division that work for negatives
ll fdiv(ll a, ll b) { return a / b - ((a % b != 0) && ((a < 0) != (b < 0))); }
ll cdiv(ll a, ll b) { return a / b + ((a % b != 0) && ((a < 0) == (b < 0))); }
// exact integer sqrt (floor)
ll isqrt(ll n) { ll r = sqrtl((ld)n); while (r * r > n) --r; while ((r + 1) * (r + 1) <= n) ++r; return r; }
// random
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll rnd(ll l, ll r) { return uniform_int_distribution<ll>(l, r)(rng); }
// anti-hack hash for unordered_map<ll, V, chash>
struct chash {
```

```
    static ull sm(ull x) { x += 0x9e3779b97f4a7c15; x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb; return x ^ (x >> 31); }
    size_t operator()(ull x) const { static const ull R = rng(); return sm(x + R); }
};
// hash a pair: use key = (ll)a << 32 | (unsigned)b
```

- Read until EOF: `while (cin >> x)`. Read a line: `getline(cin, s)`; after `cin >> n` call `cin.ignore()` (or `getline` once) before the first `getline`.
- Output many lines: use `'\n'`, never `endl` (except interactive).
- Doubles: `cout << fixed << setprecision(10) << x;` — avoid printing `-0.000000` (add `+ 0.0` or check `abs(x) < eps`).
- Sorting with comparator must be a *strict weak ordering* (`<`, never `<=`), otherwise RE/UB.
- `lower_bound(all(v), x)` → first `>= x`; `upper_bound` → first `> x`. Count of `x` in sorted v: `upper - lower`.
- `set.lower_bound(x)` is O(log n); `lower_bound(all(set), x)` is **O(n)**. Same for multiset/map.
- `multiset.erase(x)` removes **all** copies of x; use `ms.erase(ms.find(x))`.
- `priority_queue` is a max-heap; min-heap: `priority_queue<T, vector<T>, greater<T>>`.
- `accumulate(all(v), 0LL)` — the `0LL` matters.
- `(int)v.size() - 1` — `v.size() - 1` is unsigned and wraps when empty.
- `1 << k` is `int`; use `1LL << k` for k ≥ 31. `__builtin_popcountll` for `ll`.
- `memset` only for 0 / -1 / `0x3f` (int ≈ 1.06e9, sum of two still fits) .

1.2.3 Stress testing (do this after the first unexplained WA)

```
# gen.py prints a random small test; brute.cpp is the obviously-correct slow solution
g++ -O2 -o sol sol.cpp && g++ -O2 -o brute brute.cpp
for i in $(seq 1 1000); do
  python3 gen.py $i > in.txt
  ./sol < in.txt > o1.txt; ./brute < in.txt > o2.txt
  if ! cmp -s o1.txt o2.txt; then echo "DIFF on seed $i"; cat in.txt; break; fi
done
```

```
# gen.py
import random, sys
random.seed(int(sys.argv[1]))
n = random.randint(1, 6)
print(n)
print(*[random.randint(1, 10) for _ in range(n)])
```

Keep tests *tiny* (n ≤ 6, values ≤ 10): the smallest failing case is the easiest to debug by hand.

1.3 When You're Stuck

1.3.1 Constraints → target complexity

Budget ≈ 3·10^8 simple operations per second (less with maps/sets, recursion, cache misses).

n up to	target	typical techniques
8–11	O(n!·n)	all permutations (`next_permutation`), brute force
15–20	O(2^n·n), O(3^n) at 15	bitmask DP, subset enumeration, submask DP
35–45	O(2^(n/2))	meet in the middle
50–100	O(n^4), O(n^3 log)	Floyd, flows, small Hungarian, DP with 3–4 dims
300–500	O(n^3)	interval DP, Floyd, Gaussian elimination, Hungarian
2000–5000	O(n^2)	2D DP, all pairs, O(n^2) with bitset /32 for 10^4–10^5
10^5–2·10^5	O(n log n), O(n log² n), O(n√n)	sort, BIT/segtree, binary search, Dijkstra, Mo's
10^6	O(n), O(n log n) small const	linear scans, sieve, prefix sums, two pointers, KMP/Z
10^9–10^18	O(log n), O(√n), O(cube-root)	math formula, binary search, matrix power, digit DP, factorization

- **Sum of n over tests ≤ X** → per-test complexity can use X. Otherwise multiply by T!
- Two small parameters (e.g. n ≤ 10^5, k ≤ 20) → O(n·k) or O(n·2^k) DP is intended (ACPC 2022 *City Numbers*: k ≤ 20 colours on a tree).
- A weirdly specific bound is a hint: `n ≤ 15` (bitmask/interval brute), `values ≤ 200` (enumerate the value — ACPC 2018 *Dog Trouble*), `prime factors < 10^6` (trial division works), `p < 10^10, s ≤ 10^5` (s^2 can exceed p).
- Output modulo a prime → counting, combinatorics, DP; answer would be huge otherwise.

- Output a real number with 1e-6 → binary/ternary search on reals, geometry, probability.

1.3.2 Keyword → technique

if the problem says / looks like ...	try	where
minimize the maximum / maximize the minimum	binary search on answer + greedy/BFS check	§2.1
"find the smallest X such that feasible" and feasibility is monotone	binary search on X	§2.1
maximize ratio Σa/Σb	binary search λ: is max Σ(a − λb) ≥ 0 (Dinkelbach)	§2.1
unimodal function of a real variable	ternary search / golden section	§2.1
k-th smallest of something implicit	binary search on value, count ≤ mid	§2.1
subarray sums/xors, count subarrays with property	prefix sums + hashmap / two pointers	§2.2, §2.3
range add many times, query once	difference array	§2.2
sum over all pairs/subarrays of f	contribution technique: count how often each element contributes	§2.2
nearest greater/smaller, "visible", histogram	monotonic stack	§2.3
sliding window min/max	monotonic deque	§2.3
schedule jobs with deadlines/durations	sort by deadline + heap; exchange argument	§2.4
choose k things to maximize something, n large	greedy + heap; check with exchange argument	§2.4
n ≤ 40, subset sums	meet in the middle	§2.5
connectivity under edge deletions, offline	reverse time → DSU additions	§3.1
point update + range query	Fenwick / segment tree	§3.2–3.3
range update + range query	lazy segment tree	§3.3
static range min/gcd	sparse table	§3.4
k-th element in dynamic set, order of key	PBDS ordered set / BIT walk	§3.4
many range queries, offline, no updates, hard to merge	Mo's algorithm	§3.4
count / optimize over sequences, choices at each step	DP; define state = what you need to know about the past	§4.1
subsequence counting (e.g. palindromic subsequences)	interval DP with inclusion–exclusion	§4.1
interleaving two strings, edit distance, LCS	2D DP	§4.1
merge adjacent / parenthesize / split interval	interval DP O(n^3); keep max AND min if signs	§4.1
n ≤ 20 items, choose order/assignment	bitmask DP	§4.2
count numbers in [L, R] with digit property	digit DP	§4.2
tree, choose labels/colours per node with neighbour constraint	tree DP (keep best & second best)	§4.3
answer for every root	rerooting DP	§4.3
dp[i] = min_j (dp[j] + b_j·a_i)	convex hull trick / Li Chao	§4.4
dp[k][i] = min_j dp[k−1][j] + cost(j,i), cost "Monge"	divide & conquer opt / Knuth	§4.4
expected value, probability of winning, optimal stopping	linearity of expectation; DP over states backward	§4.5
turns repeat in a cycle with probabilities	geometric series / solve linear equations	§4.5
two players, optimal play, who wins	win/lose DP, Sprague–Grundy, find pattern by brute force	§4.5
shortest path, unweighted / weights 0–1 / nonneg	BFS / 0-1 BFS / Dijkstra	§5.1
shortest path with extra condition (fuel, horse, key, parity)	Dijkstra on (node, state)	§5.1
distance to nearest of many sources	multi-source BFS/Dijkstra	§5.1
path maximizing the minimum edge/vertex value	binary search + BFS, or max spanning tree, or modified Dijkstra	§5.1
which sums are representable by coins (unbounded)	Dijkstra on residues mod smallest coin	§5.1
minimum ratio spanning tree	binary search λ + MST	§5.2
dependencies / ordering	topological sort, DAG DP	§5.2
each node has one outgoing edge	functional graph: cycles + trees, binary lifting	§5.2
boolean constraints "a or b"	2-SAT	§5.3
edges whose removal disconnects	bridges / articulation points	§5.3
use every edge exactly once	Euler path (Hierholzer)	§5.3
tree path queries, ancestors, distances	LCA / binary lifting / Euler tour + BIT / HLD	§5.4

if the problem says / looks like ...	try	where
count paths in tree with property	centroid decomposition / small-to-large	§5.4
assign workers to jobs one-to-one, min cost	Hungarian / min-cost flow	§6.3
max pairs with compatibility	bipartite matching	§6.2
min #paths covering all vertices of a DAG	n − max matching (split graph)	§6.2
min #paths covering all **edges** of a DAG	min flow with lower bounds = Dilworth on edges	§6.1
cut with costs, "choose set with dependencies, max profit"	min cut / project selection	§6.1
grid with dominoes / chessboard parity	bipartite matching on black/white cells	§6.2
gcd, lcm, divisibility, a·x + b·y = c	ext. gcd, CRT	§7.1–7.2
sum of divisors, factorization of big n	sieve / trial division up to √n / Pollard rho	§7.2
count arrangements, multisets, paths	stars & bars, Catalan, inclusion–exclusion, Burnside	§7.3
linear recurrence with huge n	matrix exponentiation	§7.4
system of linear (mod 2) equations, max xor subset	Gaussian elimination / xor basis	§7.4
count pairs with sum = k for all k, pattern matching counts	FFT / NTT convolution	§7.5
string equality of substrings fast	polynomial hashing	§8.1
pattern occurrences / borders / periods	KMP prefix function, Z-function	§8.1
rotations / cyclic shifts equal	minimal rotation (Booth) as canonical form	§8.2
longest palindrome substring	Manacher	§8.2
many patterns in a text	Aho–Corasick	§8.2
distinct substrings, LCP of suffixes	suffix array + LCP	§8.2
polygon area, orientation, lattice points	shoelace, cross product, Pick	§9.1–9.2
smallest enclosing / outer boundary of points	convex hull	§9.2
interactive: judge answers queries	binary search / information-theoretic planning / gcd tricks	§10.1
evaluate expressions, mini-language	recursive descent parser	§10.2

1.3.3 Stuck-on-an-idea checklist

1. **Re-read the statement.** Underline every constraint. Is something guaranteed (distinct values, connected, sorted, a < b)? That guarantee is usually *used*.
2. **Solve tiny cases by hand** (n = 1, 2, 3) and **write a brute force**; print answers for n = 1..10 and look for a pattern (then search OEIS-style in your head: powers of 2, Catalan, Fibonacci, n^2, C(2n,n)).
3. **What is the answer monotone in?** If "answer ≥ x is feasible" is monotone → binary search. Fixing the answer often turns optimization into a yes/no greedy.
4. **Fix one thing and iterate over it:** fix the maximum element, the left endpoint, the last chosen item, the root, the value of one variable (e.g. enumerate the max time t ≤ 200 then solve the rest).
5. **Sort** the input (by start, end, value, ratio). Many greedy/DP problems become easy after sorting by the right key; find the key with an exchange argument.
6. **Reverse** something: process time backward (deletions → additions), reverse the graph, think about the complement ("at least one" = total − "none").
7. **Change what you iterate over:** over values instead of indices, over divisors instead of multiples (harmonic sum n log n), over the answer instead of the input.
8. **Model as a graph:** states = nodes, moves = edges. Grid + extra state (direction, keys, used power-up) → BFS/Dijkstra on product graph.
9. **Model as a flow/matching** when there are "pairings", "assign each X to one Y", "each used at most once", "minimum number of chains/paths".
10. **Linearity of expectation:** E[sum] = Σ E[indicator]. Never compute the full distribution if only an expectation is asked.
11. **Bound the search:** the answer is small (≤ 60 when doubling, ≤ log n), the number of distinct values is small (≤ √(2n) distinct lengths if Σ = n), only O(√n) distinct `n / i` values.
12. **Look for invariants** (parity, sum mod k, xor) in games and constructive problems.
13. Think about the **special structure:** tree (unique paths, DP), DAG (topo order), bipartite, planar grid, permutation (cycles), convex polygon ($O(n^3)$ DP over vertices).
14. Still stuck after 20–30 min with no progress → switch problems, come back later.

1.3.4 WA / RE / TLE checklist

- **Overflow**: `int * int`, sums of 10^5 values of 10^9, `1 << 40`, `INF + x`. When in doubt, `ll` everywhere.
- **Multi-test state** not reset (globals, adjacency lists, visited arrays, DSU).
- **Off-by-one**: inclusive vs exclusive ranges; 0- vs 1-indexed input; array size `n` vs `n + 1`.
- **Edge cases**: n = 1, empty set, all equal, all negative, answer 0, disconnected graph, self-loops, multi-edges, duplicate points, collinear points.
- **Reading**: multiple values per line, strings with spaces (`getline`), trailing spaces, Windows `\r`.
- **Output**: exact format/capitalisation ("Yes" vs "YES"), precision, one answer per line, `-0.0`.
- **Negative modulo**: `((a % m) + m) % m`. Subtraction in modular arithmetic: `(a - b + MOD) % MOD`.
- **Doubles compared with `==`**; use eps or exact integer arithmetic (cross products in `ll`).
- **Uninitialised** local variables / arrays (globals are zeroed, locals aren't).
- **Recursion depth** (10^6 deep DFS), **stack arrays** too big (`int a[10^7]` inside `main` → use global/vector).
- **TLE**: `endl` flushing, `map` where an array works, passing `vector` by value, O(n) `lower_bound` on `set`, `string +=` in a loop creating copies (`s = s + c` is O(n)), recomputing in loops, too many `long double`.
- **Dijkstra**: skip stale heap entries (`if (d > dist[u]) continue;`), else worst case is quadratic.
- **Comparator** not strict weak ordering → crash in `sort`.
- **Wrong algorithm**: greedy without proof → stress test against brute force.

1.4 ACPC 2016–2024: Every Problem → Technique

Kattis difficulty in brackets. Technique = the intended core idea (my analysis of the statements; `?` = likely but unverified). Use this to calibrate: what does a 4.0 vs 6.5 ACPC problem look like?

year	problem [diff]	core technique
2016	Mirror Images [1.6], Soylent [1.7], Bits [2.2]	implementation
	Wood Cutting [2.5]	sort ascending (shortest processing time first), prefix sums
	Dacey the Dice [3.3]	BFS on (cell, die orientation) — 24 orientations
	Flower Garden [3.6]	prefix of Manhattan distances + sieve for primes
	Palindromic Naming [4.1]	count palindromic subsequences, interval DP $O(n^2)$ mod p (§4.1)
	City Destruction [5.6]	linear DP over buildings with state "was neighbour destroyed before me"
	Modelling Problems [6.8]	parse a mini-language, brute force over unknown 0/1 values
2017	Musical Scales [1.6], Tetration [1.7]	implementation / math (x = a^(1/a))
	Quantum Superposition [2.2]	DAG DP: set of reachable path lengths (bitset), combine two sets
	Anthony and Diablo [2.3]	geometry: max area for perimeter is a circle
	Divide by 100 [3.2]	big-number string manipulation
	Concentration [4.8]	probability / game simulation reasoning ?
	Lane Switching [5.5]	maximize the minimum (bottleneck path) over intervals: Dijkstra/BFS ?
	Maximal Sequences [6.2]	offline by start index; ordered set of next occurrences per value; check first m+1
	Matchings [7.5]	2-D pattern matching counts via FFT convolution, one FFT per colour (§7.5)
2018	Early Winter [2.0], Eating Out [2.5]	implementation / pigeonhole counting
	Arachnophobia [4.0]	multi-source Dijkstra from spiders + binary search D + Dijkstra s→t
	Dog Trouble [5.8]	enumerate max time t (≤ 200), assignment (Hungarian) maximizing Σt_i (§6.3)
	PUBNite [5.8]	geometry + binary/ternary search on time
	Exploding Kittens [6.4]	simulation, skipping irrelevant cards ?
	Homework [6.5]	string interleaving DP O(len1·len2) (bitset if tight)
	Trimming Convex Polygons [6.9]	DP over convex polygon vertices $O(n^3)$ (fix first kept vertex, triangle fan)

year	problem [diff]	core technique
2020	Test Drive [1.9], Laptop Stickers [1.9], Warring Scoring [2.8]	implementation
	Path Crossings [2.9]	hashing (x,y,t) / sort events
	Straights [2.9]	greedy on value counts
	Listen To Your Boss [3.9]	tree ancestor queries: Euler tour tin/tout (§5.4)
	Wormhole Extreme!!! [4.4]	kinematics; minimum initial velocity via v^2 recursion / binary search
	Hired Help [5.5]	binary search #employees + greedy deadline check
	Monochromatic Minesweeper [6.2]	constructive / search with pruning ?
	Password Rotation [6.3]	canonical form = min rotation of s and reverse(s) (Booth), hash set (§8.2)
	Wrapping Trees [7.3]	fractal → convex hull of the self-similar set; geometry
	Best Investing [7.8]	math / greedy over deposit–withdraw schedule ?
2021	Finding An A [1.2], GCVWR [1.4], Square Peg [1.5], Simone [1.8], Colour Wars [2.3]	implementation
	Snakes and Masters [2.9]	Fibonacci DP mod 10 ^ 6
	Bitmask [3.1]	string/bit matching with DP ?
	Menu Updates [3.3]	maintain smallest available number: set/heap
	Associative Exponents [3.4]	compare a ^ (b ^ c) vs (a ^ b) ^ c ⇔ b ^ c vs b·c, careful with overflow
	Ghostbusters 3 [3.8]	non-crossing assignment: sort + greedy/DP ?
	Wonky Pizza [4.2]	polar area $\int r^2/2\, d\theta$ closed form + binary search the angle
	Beep Code [4.6]	bit decomposition of a sum of square waves; reason level by level
	Balls and Bins [5.1]	optimal-stopping expected-value DP over (#bins with 1, #bins with 2) (§4.5)
	Swapping Places [8.1]	constructive: permutation routing on spanning tree, process leaves first
	Compression [8.2]	DP over block boundaries with careful cost precomputation
2022	Undead or Alive [1.3], Ordinals [1.8], Count Doubles [1.8]	implementation
	Contingency Planning [2.1]	$\Sigma_k\ n!/(n-k)!$ with cap
	Simple Factoring [2.4]	discriminant $b^2 - 4ac$ is a perfect square
	Maximum Fix [3.4]	each element votes for the one shift that fixes it; count array
	Supply Routes [3.5]	offline: reverse edge deletions → DSU unions (§3.1)
	Fight to Survive [3.8]	greedy / simulation ?
	City Numbers [4.4]	tree DP dp[v][colour], keep best & second best child value (§4.3)
	Long Walk [5.5]	implicit graph i ↔ 2i mod 2 ^ n: bit manipulation, LCA-like on binary strings
	Braaaains [5.8]	Frobenius/coin reachability: Dijkstra on residues mod smallest size (§5.1)
	Tricky Factoring [6.7]	factor a, c (primes < 10 ^ 6), enumerate divisor pairs, count distinct b = dg + ef
	Maximal Parentheticals [8.0]	interval DP over expression tracking max and min (signs flip)
	Escape Routes [8.3]	min #paths covering all DAG edges = min flow with lower bounds (§6.1)
2023	Buildin' Fences [1.5]	implementation
	Flying Straight [3.0]	trigonometry (arc vs tangent)
	Buildin' Roads [3.3]	minimum ratio spanning tree: binary search λ + MST (§2.1, §5.2)
	OBD Codes [3.9]	pattern matching on event streams ?
	Counting Cacti [4.0]	grid parsing + flood fill components

year	problem [diff]	core technique
	Outlaw Numbers [4.0]	aliquot sequence: sum of proper divisors via factorization, ≤ 32 steps
	Wrangler Rating [4.5]	careful simulation of rules
	Circuit Game [5.2]	graph connectivity under switch removal + budget ?
	Dice Tables [5.2]	counting multisets (stars and bars) over overlapping face ranges, mod 10^9+9
	Desert Strike [5.4]	lattice counting of a rasterized rotated square (math)
	High Noon Language Showdown [5.5]	brute force over n with exact integer ceil(log2)
	Temporal Turmoil [6.7]	clock-hand alignment math with exact fractions
	Horse Snatchers [6.8]	binary search horse speed + Dijkstra on (node, has horse)
	Gunslingers [7.8]	probability DP over survivor sets with cyclic turns (geometric series)
2024	Snowfall [1.6], Melting Snow [1.7], Handheld Fan [2.2]	implementation
	Wrapping Snowballs [3.5]	?
	Alpine Agility [3.8]	grid DP (moves S/E/SE)
	Slippery Floor [3.8]	BFS where a move slides until hitting an obstacle
	Topographic Isolation [4.0]	monotonic stack: nearest ≥ on each side (§2.3)
	A Little to the Right [4.2]	count distinct sorted orders among properties with all-distinct values
	Hockey Fans [4.4]	binary search on D + greedy left-to-right placement
	Uniform Variation [4.7]	brute force: groups of all-different are ≤ 4; enumerate small subsets
	Snow Way Out [6.7]	interactive, 6 comparisons on 10×10 grid: offline minimax search for a strategy
	Four Questions [7.1]	interactive: $s^2 \bmod p \Rightarrow p$ divides $s^2 - r$; gcd of such values (§10.1)
	Speedy Slopes [7.2]	shortest path with speed state on grid ?

Patterns to take away

- Binary search on the answer appears almost every year (Arachnophobia, Hired Help, Hockey Fans, Horse Snatchers, Buildin' Roads).
- Dijkstra variants: multi-source, state-augmented, residue graph (Braaaains).
- Probability/expected value DP appears most years at 5–8 difficulty.
- One flow/matching problem every year or two (Dog Trouble, Escape Routes).
- Interactive problems appeared in 2024 (two of them) — read §10.1 before the contest.
- Number theory with "all prime factors < 10^6" or big-number twists.